



Programmer's Manual

Table of Contents

Table of Contents.....	1
R1 Functions.....	4
void version(void).....	4
void help(void).....	4
void help_extra(char *arg).....	4
int shutdown(void).....	5
char *get_date(void).....	5
char *get_time(void).....	5
void set_date(char[] date).....	5
void set_time(char[] time).....	5
void commhand(void).....	6
int bcd_to_int(char number).....	6
char int_to_bcd(int number).....	6
int kernel_get_day(void).....	6
int kernel_get_month(void).....	7
int kernel_get_year(void).....	7
int kernel_get_hour(void).....	7
int kernel_get_minute(void).....	7
int kernel_get_second(void).....	7
void kernel_set_time_second(int second).....	7
void kernel_set_time_minute(int minute).....	8
void kernel_set_time_hour(int hour).....	8
void kernel_set_time_day(int day).....	8
void kernel_set_time_month(int month).....	8
void kernel_set_time_year(int year).....	8

int serial_poll(device dev, char* buffer, size_t len).....	8
void arrow_handler(char* buffer, int* cursor, size_t* bytesRead, device dev, char starting_byte);.....	9
void delete_handler(char* buffer, size_t* bytesRead, int* cursor, int* is_DEL).....	9
R2 Functions.....	11
void startup_sequence().....	11
char *itoa(int value).....	11
void set_color(char *color).....	11
reset_color().....	11
void print(char *str).....	11
void print_err(char *str).....	11
void print_suc(char *str).....	11
void print_char(char c).....	12
void print_ws(int count).....	12
void print_hr(void).....	12
void print_col(char *str, char *color).....	12
void clear(void).....	12
void fifo_node(Node **head, Node **tail, Node *node).....	12
void priority_node(Node **head, Node **tail, Node *node).....	12
struct pcb* pcb_find(const char *name).....	13
void pcb_insert(struct pcb* name).....	13
int pcb_remove(struct pcb* name).....	13
int create_pcb(char *name, char *class, int priority).....	13
int delete_pcb(char *name).....	13
int block_pcb(char *name).....	13
int unblock_pcb(char *name).....	14
int suspend_pcb(char *name).....	14
int resume_pcb(char *name).....	14
int set_pcb_priority(char *name, int priority).....	14
int pcb_validate_name(char *name).....	14
int show_pcb(char *name).....	14
void show_ready(void).....	15
void show_blocked(void).....	15
void show_all(void).....	15
void pcb_print_queues(int print_R, int print_RS, int print_B, int print_BS).....	15
void print_pcb(pcb* currpcb).....	15
struct pcb* pcb_setup(char * process_name, int class, int priority).....	15
struct pcb* pcb_allocate(void).....	16
int pcb_free(struct pcb*).....	16
char* strcpy(char destination[], char source[]);.....	16
char* get_exec_state(int state).....	16

char* get_disp_state(int state).....	16
char* get_class_state(int state).....	16
R3 Functions.....	17
struct context* sys_call(struct context *current_process).....	17
pcb* pop(struct pcb** queue_head).....	17
void push_by_priority(pcb** queue_head, pcb** queue_tail, struct pcb* process).....	17
pcb* create_pcb_no_insert(char *name, char *class, int priority).....	18
void insert_context(pcb* process, void(*func)(void)).....	18
void load_r3(int disp_state).....	18
void load_process(char* name).....	18
void resume_all(void).....	18
int count_expected_range(int param_count, int min, int max).....	18
Int count_expected(int param_count, int expected_count).....	19
void print_param_bounds_error_range(int param_count, int expected_min, int expected_max, char* command_name).....	19
void print_param_bounds_error(int param_count, int expected_count, char* command_name).....	19
R4 Functions.....	19
void mini_startup(void).....	19
struct alarm* alarm_create(char *msg, int h, int m, int s).....	19
int alarm_add(char *msg, int h, int m, int s).....	20
void alarm_check(void).....	20
void running_alarm(void).....	20
void alarm_remove(alarm *a).....	20
int set_alarm(char time[16], char *message).....	20
R5 Functions.....	20
int alloc_mem(int size).....	20
int free_mem(char *block_addr).....	21
void print_mcbs(int flag).....	21
void print_mcb(mcb* curr_mcb).....	21
void initialize_heap(size_t size).....	21
void* allocate_memory(size_t size).....	21
int free_memory(void* addr).....	21
char *itoh(int dec_num).....	22
int htoi(char *hex_num).....	22
int int_of_hex(char digit).....	22
char hex_of_int(int num).....	22
int power_int(int base, int power).....	22
R6 Functions.....	23
int serial_open(device dev, int speed).....	23
int serial_close(device dev).....	23

int serial_read(device dev, char* buf, size_t len).....	23
int serial_write(device dev, char *buf, size_t len).....	23
void serial_interrupt(void).....	23
void serial_input_interrupt(dcb* dcb).....	24
void serial_output_interrupt(dcb* dcb).....	24
int has_unread_char(dcb* device).....	24
char ring_read(dcb* device).....	24
void ring_write(dcb* device, char input).....	24
int loop_ring(int place, int length).....	24
void enqueue_iocb(dcb *d, struct iocb *new_iocb).....	25
iocb* dequeue_iocb(dcb *d).....	25
dcb* find_dcb(device dev).....	25
int dcbArrLoc(device dev).....	25
void IOScheduler(iocb* queued_iocb).....	25
void IOComplete(void).....	25
iocb* create_iocb(int op, dcb *curr_dcb, char *buffer, int size).....	26
int store_char(char* buffer, int len, char single_char).....	26

R1 Functions

void version(void)

Author(s): Elias Kittle

Description: Runs when the version command is input. The function writes the version constant and __DATE__ compiler macro to the terminal. The version constant must be updated manually.

void help(void)

Author(s): Elias Kittle

Description: Runs when the help command is input. Writes to the terminal every command and argument with a brief description of what they do.

Function must be updated after new commands or arguments are added.

void help_extra(char *arg)

Author: Mason Phillips

Description: This is a helper function that allows an additional argument to be provided to the help command. It is called after the commhand checks that the help function is called with an additional argument. If said argument is a valid command, it uses sys_req(WRITE, ...) to provide an in-depth explanation of said function, including all arguments and their expected formats.

int shutdown(void)

Author(s): Elias Kittle, Mason Phillips

Description: This function is called when the user types the shutdown command. It then prompts for confirmation using `sys_req(READ, ...)`, and reads only the next byte. If the byte is the character 'y', the system continues with shutdown and exits out of command handler. If the byte is anything else, it exits and returns control to the command handler.

char *get_date(void)

Author(s): Blake Mongold

Description: It starts by calling 3 kernel functions, `kernel_get_day`, `kernel_get_month`, `kernel_get_hour`. It then saves what those 3 kernel functions return into `int` `day`, `month`, and `year`. After that the century is added onto the year, since `kernel_year` calls only return the decade and year. A `char` array is created that is used to break up the day, month and year, to display it properly to the user. Each `int` has a + '0' added to it in order to convert from `int` to `char`. Then the array is finished off by creating a new line and then ending it. Finally it returns a pointer to the array.

char *get_time(void)

Author(s): Blake Mongold

Description: It starts by calling 3 kernel functions, `kernel_get_second`, `kernel_get_minute`, `kernel_get_hour`. It then saves what those 3 kernel functions return into `int` `second`, `minute`, and `hour`. A `char` array is created that is used to break up the second, minute and hour, to display it properly to the user. Each `int` has a + '0' added to it in order to convert from `int` to `char`. Then the array is finished off by creating a new line and then ending it. Finally it returns a pointer to the array.

void set_date(char[] date)

Author(s): Blake Mongold

Description: Three `ints` are created to store each portion of the date, `int` `day`, `month`, and `year`. Then there is a pointer called `tok` that calls `strtok` that breaks up the input between each / and stores each in their respective `int` variable. It then makes sure the day and month entered by the user are correct. Then the year is broken in half, with the millennium and century stored in the `century` variable and the decade and year stored back into the `year` variable. Then it calls `kernel_set_day`, `kernel_set_month`, and `kernel_set_year` to store the input in the RTC. After it is successful it returns 0.

void set_time(char[] time)

Author(s): Blake Mongold

Description: Three ints are created to store each portion of the time, int second, minute, and hour. Then there is a pointer called tok that calls strtok that breaks up the input between each : and stores each in their respective int variable. It then makes sure the second, minute, and hour entered by the user are correct. Then it calls kernel_set_second, kernel_set_minute, and kernel_set_hour to store the input in the RTC. After it is successful it returns 0.

void commhand(void)

Author(s): Mason Phillips

Description: This function is the main driver of the project, as it infinitely prompts for and handles user input until the mpx is shutdown. It prompts for user input by calling sys_req(READ, ...), which calls serial_poll() to receive input from the keyboard. It tokenizes the buffer returned from sys_req(READ, ...) based on space characters, using strtok(). The first token is interpreted as the command, while any following tokens are considered arguments. Commhand compares the first token to the list of available commands, and calls the corresponding command function if it is valid, and the correct number of arguments are provided. If there are any issues with either the number of arguments provided or the command provided, a descriptive error message is printed using sys_req(WRITE, ...). After either outcome, the commhand reenters the loop, prompting for user input once again.

int bcd_to_int(char number)

Author(s): Blake Mongold

Description: This function converts bcd to int. It starts by shifting the first 4 binary numbers on the left to right by 4. This gets rid of the 4 binary numbers on the right. Then you & compare it to 0x0F, which is 0000 1111 in binary. This makes sure that the 4 left most bits are always 0 and the 4 right most bits 1 or 0 based on the received bcd. Then the result is saved in the int ten_digit. The same is done for the int one_digit, but instead of shifting it to the right, you only compare it to 0x0F. Then it returns the result of multiplying int ten_digit by 10 and adding int one_digit to finish off the conversion.

char int_to_bcd(int number)

Author(s): Blake Mongold

Description: It takes the number provided and breaks it into int ten_digit and int one_digit. Ten_digit takes the number and divides it by ten and the one_digit takes the number modulo by 10. Then it returns the result of moving ten_digit to the left 4 and | comparing it to the one_digit.

int kernel_get_day(void)

Author(s): Blake Mongold

Description: It uses outb to write to the index register 0x70 from the port 0x07, that has the day value. It then uses inb to read from data register 0x71 that has the value from outb stored in it. It then calls the function bcd_to_int to convert and return it.

int kernel_get_month(void)

Author(s): Blake Mongold

Description: It uses outb to write to the index register 0x70 from the port 0x08, that has the month value. It then uses inb to read from data register 0x71 that has the value from outb stored in it. It then calls the function bcd_to_int to convert and return it.

int kernel_get_year(void)

Author(s): Blake Mongold

Description: It uses outb to write to the index register 0x70 from the port 0x09, that has the year value. It then uses inb to read from data register 0x71 that has the value from outb stored in it. It then calls the function bcd_to_int to convert and return it.

int kernel_get_hour(void)

Author(s): Blake Mongold

Description: It uses outb to write to the index register 0x70 from the port 0x04, that has the hour value. It then uses inb to read from data register 0x71 that has the value from outb stored in it. It then calls the function bcd_to_int to convert and return it.

int kernel_get_minute(void)

Author(s): Blake Mongold

Description: It uses outb to write to the index register 0x70 from the port 0x02, that has the minute value. It then uses inb to read from data register 0x71 that has the value from outb stored in it. It then calls the function bcd_to_int to convert and return it.

int kernel_get_second(void)

Author(s): Blake Mongold

Description: It uses outb to write to the index register 0x70 from the port 0x00, that has the second value. It then uses inb to read from data register 0x71 that has the value from outb stored in it. It then calls the function bcd_to_int to convert and return it.

void kernel_set_time_second(int second)

Author(s): Blake Mongold

Description: It uses outb to write to the index register 0x70 from the port 0x00, that has the second value. Then it calls youtube again to overwrite the value in data register 0x71 with the value provided. Before it is called though, int_to_bcd is called to convert it into a bcd.

void kernel_set_time_minute(int minute)

Author(s): Blake Mongold

Description: It uses outb to write to the index register 0x70 from the port 0x02, that has the second value. Then it calls youtube again to overwrite the value in data register 0x71 with the value provided. Before it is called though, int_to_bcd is called to convert it into a bcd.

void kernel_set_time_hour(int hour)

Author(s): Blake Mongold

Description: It uses outb to write to the index register 0x70 from the port 0x04, that has the second value. Then it calls youtube again to overwrite the value in data register 0x71 with the value provided. Before it is called though, int_to_bcd is called to convert it into a bcd.

void kernel_set_time_day(int day)

Author(s): Blake Mongold

Description: It uses outb to write to the index register 0x70 from the port 0x07, that has the second value. Then it calls youtube again to overwrite the value in data register 0x71 with the value provided. Before it is called though, int_to_bcd is called to convert it into a bcd.

void kernel_set_time_month(int month)

Author(s): Blake Mongold

Description: It uses outb to write to the index register 0x70 from the port 0x08, that has the second value. Then it calls youtube again to overwrite the value in data register 0x71 with the value provided. Before it is called though, int_to_bcd is called to convert it into a bcd.

void kernel_set_time_year(int year)

Author(s): Blake Mongold

Description: It uses outb to write to the index register 0x70 from the port 0x09, that has the second value. Then it calls youtube again to overwrite the value in data register 0x71 with the value provided. Before it is called though, int_to_bcd is called to convert it into a bcd.

int serial_poll(device dev, char* buffer, size_t len)

Author(s): Michael Kaulfuss

Description: This function returns the number of bytes read from the terminal that the user inputs and polls for serial input. The method starts by initializing a cursor and bytesRead variable. Cursor represents the current position of the cursor. bytesRead represents the number of bytes that are currently being displayed by the command line and being stored in the buffer. The first check that is performed is to see that the bytesRead does not exceed the length (len) of the buffer to prevent overflow. The first byte is read in with the inb command from the port

with the line status register (LSR) as an offset. Lowercase lsr stores the result of this in a char variable which then uses bitwise operations on the last bit to see if the data is ready to be processed. If it is, a few checks are performed which in turn may call helper functions. The first check checks if the user hit enter. If they did, it ends the buffer at bytesRead with a null terminator then returns with a new starting line. The next check sees if the user input backspace with the ASCII hex for two possible backspace cases. A variable named is_DEL is created, and the int represents 0 being false and 1 being true. The delete_handler function is called inside this. The next check sees if the current byte is ESC which could possibly be part of a key press that is more than one bit. If the byte is ESC, the arrow handler function is called. The final check is a comprehensive one that sees if the byte read is alphanumeric or a symbol which all fall within a certain range. Inside this loop for a for if the cursor is positioned in any place besides the front the statement the loop executes and shifts every value to the right by one index till it reaches the cursor position which it then stores the byte in the position of the cursor inside the buffer. The cursor position and number of bytesRead increases. The next step is to output the contents of the buffer that has been updated and change and output the changes to the cursor position as well. After the UI is no longer receiving input, the buffer is ended with a null terminator at bytes and returns the number of bytes read to the user.

```
void arrow_handler(char* buffer, int* cursor, size_t* bytesRead,  
device dev, char starting_byte);
```

Author(s): Michael Kaulfuss

Description:

The arrow handler is used for the long ESC sequence that comes in multiple bytes from the arrow keys. When this helper function is called, a char array is instantiated that stores 4 bytes starting with the starting byte from the polling function. The second and third byte are stored when the byte is available and the RBR portion reads the actual byte. This is repeated one more time with the next byte being stored. If the next char in the sequence is a bracket check every letter case A B C D. If it is C or D this is the right and left arrow key respectively. For the C case, it checks that the cursor doesn't extend beyond the content of the buffer and visually displays where the cursor is. The cursor is incremented. For case D, it ensures that the cursor cannot go before the 0th index of the buffer. The outb function moves the cursor back a single index visually and finally the cursor is decremented since the position changed. The final case sees if the byte is 3. This would potentially make it part of the delete sequence. If it was pressed another byte is needed to verify that the next byte is tilde. The inb while loop function is called again for the final byte. If tilde was pressed, an int is created with is_DEL = 1 which means true. The delete_handler function is called.

```
void delete_handler(char* buffer, size_t* bytesRead, int* cursor,  
int* is_DEL)
```

Author(s): Michael Kaulfuss

Description: This helper function processes whether the delete key was pressed or backspace was pressed. The first case checked by the function sees if the `is_DEL` parameter is false and if the cursor's position is 0. If it is there is nothing to delete. The next case checks if the delete key wasn't pressed and the cursor is ahead in ahead or equal. The last case is solely for whether the backspace was pressed. The `outb` function moves the cursor back a single byte and decrements the cursor position which blanks the last position and moves the cursor back. The next for loop rebuilds the buffer with items shifted in front of the cursor to the left. `bytesRead` is decremented since something was deleted from the buffer. This is used for delete as well since delete performs on char deletion. The last two for loops reposition the cursor and redisplay the modified buffer to the user.

R2 Functions

`void startup_sequence()`

Author(s): Mason Phillips

Description: Prints a flashy ASCII art of the OS logo.

`char *itoa(int value)`

Author(s): Mason Phillips

Description: Takes an integer as input and converts it to a string character by character.

`void set_color(char *color)`

Author(s): Mason Phillips

Description: Passes a color escape sequence defined in color.h, and sets the current terminal text color to the given escape sequence, simply by printing it.

`reset_color()`

Author(s): Mason Phillips

Description: Uses the terminal default text color escape sequence, and sets the color to said color by printing the sequence.

`void print(char *str)`

Author(s): Mason Phillips

Description: Calls `sys_req(WRITE, ...)` with `strlen()` of the passed string as the size parameter, printing the given string to the terminal.

`void print_err(char *str)`

Author(s): Mason Phillips

Description: Just like `print`, uses `sys_req(WRITE, ...)` and `strlen()` to print text to the terminal . Instead of using the current text color, it sets the text color to red, prints, then resets the color.

`void print_suc(char *str)`

Author(s): Mason Phillips

Description: Just like `print`, uses `sys_req(WRITE, ...)` and `strlen()` to print text to the terminal . Instead of using the current text color, it sets the text color to green, prints, then resets the color.

void print_char(char c)

Author(s): Mason Phillips

Description: Prints a singular character to the terminal using sys_req(WRITE, ...).

void print_ws(int count)

Author(s): Mason Phillips

Description: Prints the number of whitespace characters corresponding to count.

void print_hr(void)

Author(s): Mason Phillips

Description: Prints a horizontal rule by looping using print_char('-').

void print_col(char *str, char *color)

Author(s): Mason Phillips

Description: Prints the given string, first setting the color to the given color constant from color.h, then resetting the color after printing.

void clear(void)

Author(s): Mason Phillips

Description: Clears the screen by printing two escape characters. One to move the cursor to the top left corner, the other to clear the screen from the cursor down.

void fifo_node(Node **head, Node **tail, Node *node)

Author(s): Blake Mongold

Description: Sorts the node in First Come First Out (FIFO) doubly linked list. Checks to see if there is a head node, if not then the inserted node is attached to both the head and tail pointer. If there is a head node, then the inserted node is inserted at the end of the queue.

void priority_node(Node **head, Node **tail, Node *node)

Author(s): Blake Mongold

Description: Sorts the node in a priority doubly linked list. Lower numerical priority represents a higher priority (0=highest priority). It checks to see if the queue is empty, if so then the head and tail pointers are attached to the inserted node. Then a while loop runs until it goes through the whole queue or it finds a place in the queue where the inserted node's priority is higher than the priority of the current node in the queue. It is then inserted into the queue and returns.

struct pcb* pcb_find(const char *name)

Author(s): Blake Mongold

Description: Searches all four process queues (Ready, Ready Suspended, Blocked, and Blocked Suspended) for a PCB with a name matching the provided string. Each node in the queue is examined and the PCB name is compared to the input string. If a matching PCB is found, a pointer to that PCB is returned. If no matching PCB exists in any queue, the function returns NULL.

void pcb_insert(struct pcb* name)

Author(s): Blake Mongold

Description: Starts by creating a Node structure and storing the PCB parameters in it. Then depending on the execution state, it gets sent to either fifo_node() or priority_node() to be inserted into the correct queue. After that it returns.

int pcb_remove(struct pcb* name)

Author(s): Blake Mongold

Description: Removes the PCB from its current queue. The queue is determined by its execution state. Once found, the surrounding nodes previous and next pointers are updated. If it is successful then it returns 0, if it can't find the state then it returns -2, and if it can't find the node then it returns -1.

int create_pcb(char *name, char *class, int priority)

Author(s): Mason Phillips

Description: Provides error checking for inputs, then creates a new pcb by calling pcb_setup() with the given parameters. If any of the given parameters are invalid, a descriptive error message will be printed. After creating the pcb, it is inserted into the correct queue using pcb_insert().

int delete_pcb(char *name)

Author(s): Mason Phillips

Description: Provides error checking for name, then deletes the PCB with the provided name. If the name is invalid or the pcb does not exist, a descriptive error message will be printed. Otherwise, the pcb is removed using pcb_remove(), and its memory is freed using pcb_free().

int block_pcb(char *name)

Author(s): Mason Phillips

Description: Blocks a pcb, first checking that the name is valid, pcb exists, and that it is not already blocked. If all true, the pcb is removed from its current queue using `pcb_remove()`, its execution state is set to `BLOCKED`, and it is reinserted using `pcb_insert()`.

`int unblock_pcb(char *name)`

Author(s): Mason Phillips

Description: Unblocks a pcb, first checking that the name is valid, pcb exists, and that it is not already unblocked. If all true, the pcb is removed from its current queue using `pcb_remove()`, its execution state is set to `UNBLOCKED`, and it is reinserted using `pcb_insert()`.

`int suspend_pcb(char *name)`

Author(s): Mason Phillips

Description: Suspends a pcb, first checking that the name is valid, pcb exists, and that it is not already suspended. If all true, the pcb is removed from its current queue using `pcb_remove()`, its dispatch state is set to `SUSPENDED`, and it is reinserted using `pcb_insert()`.

`int resume_pcb(char *name)`

Author(s): Mason Phillips

Description: Resumes a pcb, first checking if name is valid, pcb exists, and that it is not already not suspended. If all true, the pcb is removed from its current queue using `pcb_remove()`, its dispatch state is set to `NOT_SUSPENDED`, and it is reinserted using `pcb_insert()`.

`int set_pcb_priority(char *name, int priority)`

Author(s): Mason Phillips

Description: Updates a PCB's priority to the given priority after checking inputs. First it validates that the PCB name is valid, that it exists, and that priority is within the range 0-9. Once all is confirmed, the PCB is removed using `pcb_remove()`, the priority field is set to the passed int, and the PCB is reinserted using `pcb_insert()`.

`int pcb_validate_name(char *name)`

Author(s): Mason Phillips

Description: Helper function to ensure a passed string is a valid PCB name, at most 8 characters long.

`int show_pcb(char *name)`

Author(s): Elias Kittle

Description: Runs when the `show_pcb` command is input. It calls `pcb_find` with the provided name. Prints "pcb_found" alongside calling `print_pcb` or if not found prints an error.

void show_ready(void)

Author(s): Elias Kittle

Description: Runs when the show_ready command is input. Calls pcb_print_queues with print_R and print_RS arguments set to 1, or true, to print out the ready and ready suspended queues.

void show_blocked(void)

Author(s): Elias Kittle

Description: Runs when the show_blocked command is input. Calls pcb_print_queues with print_B and print_BS arguments set to 1, or true, to print out the blocked and blocked suspended queues.

void show_all(void)

Author(s): Elias Kittle

Description: Runs when the show_all command is input. Calls pcb_print_queues with all variables set to 1, or true, to print out every pcb queue.

void pcb_print_queues(int print_R, int print_RS, int print_B, int print_BS)

Author(s): Elias Kittle

Description: Arguments print_R, print_RS, print_B, and print_BS, if the value is 1, print the ready, ready suspended, ready blocked, and blocked suspended queues respectively. Each loop starts at their respective head node and continues to call print_pcb with every following node in the queue until the next node is null.

void print_pcb(pcb* currrpcb)

Author(s): Elias Kittle

Description: Prints the provided pcb's name, class, priority, execution status, and suspend status. Calls itoa to convert the priority number to a string and get_class, get_exec_state, and get_disp_state to convert the respective fields to string.

struct pcb* pcb_setup(char * process_name, int class, int priority)

Author(s): Michael Kauffuss

Description: The main purpose of this function is to store information input into the parameters into the instance of a struct. To get started the pcb_allocate function makes the function. It then stores data about class, and priority simply into the struct. For the process_name a loop is implemented to get all chars from in the process_name and store it into the pcb struct. If the end

of the process_name is reached the char at the index is ended by a null terminator, the loop is broken and finally the struct is returned. If the maximum number of characters for the process name is reached, the last index is ended with a null terminator and the struct is returned.

struct pcb* pcb_allocate(void)

Author(s): Michael Kaulfuss

Description: This function creates a new instance of a pcb struct and initializes data members. It starts by allocating memory for a pcb using the sys_alloc_mem function given. A check to see whether the function properly allocated the memory is given where a return value of NULL is for unsuccessful data allocation. Otherwise, the program fills the provided stack with null terminators so as to not leave gaps in new data.

int pcb_free(struct pcb*)

Author(s): Michael Kaulfuss

Description: It frees all associated memory with a struct. If the pcb is invalid it returns with a -1 as the struct is no longer valid. Otherwise, the sys_free_mem function is called and the struct is freed from memory.

char* strcpy(char destination[], char source[]);

Author(s): Michael Kaulfuss

Description: Copies characters from two strings char by char until a null terminator is reached by the source. Otherwise, a null terminator ends the destination character array. Returns a pointer to the destination character array.

char* get_exec_state(int state)

Author(s): Elias Kittle

Description: Used to get the string version of the pcb execution enum by having a switch case that returns string literals based on what the state integer is equal to. Returns an "invalid" char* message if the state is not found.

char* get_disp_state(int state)

Author(s): Elias Kittle

Description: Used to get the string version of the pcb dispatch enum by having a switch case that returns string literals based on what the state integer is equal to. Returns an "invalid" char* message if the state is not found.

char* get_class_state(int state)

Author(s): Elias Kittle

Description: Used to get the string version of the pcb class enum by having a switch case that returns string literals based on what the state integer is equal to. Returns an "invalid" char* message if the state is not found.

R3 Functions

`struct context* sys_call(struct context *current_process)`

Author: Michael Kaulfuss

Description: Used primarily in context switching. Relies on the EAX register to dictate which processes are yielded to the CPU for sys_req() to process based on saved states. This function uses an initialized pcbPtr to store the actively executing process and a contextPtr to dictate when this function was first entered. IDLE and EXIT processes are handled respectively and all other values on the EAX register store a value of -1. This struct relies on queues created in previous modules where two helper functions are used. For IDLE processes, the most basic case is when a value is popped from the front of the queue and the global pcbPtr is enqueued. The pcb then has its register assigned to 0 to be used for the EXIT condition if there is another call for this pcb. For the EXIT condition, the global pcbPtr for the actively executing process is freed, the first value from the ready queue is popped off, if there is nothing in the ready queue, then the contextPtr's EAX is set to 0.

`pcb* pop(struct pcb** queue_head)`

Author: Michael Kaulfuss

Description: Removes a single PCB from a doubly linked list or unofficial queue. Start by checking if there is anything in the queue by checking the head. Then, the queue makes a new pcb called, "temp" to store the value that we need to return. The head of the queue then points to the next value temp is pointing to. The previous pointer to the queue's head is then set to null. Temp is then fully detached by setting its next and previous ptr to null. Temp is finally returned.

`void push_by_priority(pcb** queue_head, pcb** queue_tail, struct pcb* process)`

Author: Michael Kaulfuss & Mason Phillips

Description: This function enqueues a process at the correct priority location of a doubly linked list (our ready queue). We first start by modifying a node that was passed in. The ptr to the next value is set to NULL since we want to instantiate the next data member. The previous val for the PCB is set to the queue_tail as well. A similar instance, like in the pop function, of accounting for an empty queue is done again. The next process for the queue tail is the value assigned which points to the now fully connected PCB to the doubly linked list. Finally the queue tail takes over the newly inserted PCB's address with assignment and finishes this function

`pcb* create_pcb_no_insert(char *name, char *class, int priority)`

Author: Blake Mongold

Description: Provides error checking for inputs, then creates a new pcb by calling `pcb_setup()` with the given parameters. If any of the given parameters are invalid, a descriptive error message will be printed. After creating the pcb, it returns a pointer to the created pcb.

`void insert_context(pcb* process, void(*func)(void))`

Author: Blake Mongold

Description: Initializes the execution context for the given process so that it begins running at the specified function. This includes setting up the process's stack and initial register values so the dispatcher can correctly start the process.

`void load_r3(int disp_state)`

Author: Blake Mongold

Description: Creates and loads the R3 test processes into the system. Each process is initialized with a context and placed into the appropriate queue so it can be scheduled and executed based on the given dispatch state.

`void load_process(char* name)`

Author: Blake Mongold

Description: Creates and loads a single process using the provided name. The process is initialized, given a context, and inserted into the ready queue so it can be dispatched.

`void resume_all(void)`

Author: Blake Mongold

Description: Changes all suspended processes back to a non-suspended state. Any processes that were suspended in either the ready or blocked queues are moved to the appropriate non-suspended queue.

`int count_expected_range(int param_count, int min, int max)`

Author: Elias Kittle

Description: Returns 1 if the count was in the expected range. Returns 0 if it wasn't. Used for `commhand.c`'s commands that expect a range of possible parameter counts.

`int count_expected(int param_count, int expected_count)`

Author: Elias Kittle

Description: Returns 1 if the count was the expected amount. Returns 0 if it wasn't. Used for commhand.c's commands that expect a specific parameter count.

`void print_param_bounds_error_range(int param_count, int expected_min, int expected_max, char* command_name)`

Author: Elias Kittle

Description: Prints an out of bounds parameter range error using the information of the arguments. Used for commhand.c's commands that expect a range of possible parameter counts.

`void print_param_bounds_error(int param_count, int expected_count, char* command_name)`

Author: Elias Kittle

Description: Prints an out of bounds parameter error using the information of the arguments. Used for commhand.c's commands that expect a specific parameter count.

R4 Functions

`void mini_startup(void)`

Author: Mason Phillips

Description: Prints a smaller version of the previous startup screen in order to remain within the 1024 stack bytes allocated to processes. Used in the commhand.c before the commhand loop.

`struct alarm* alarm_create(char *msg, int h, int m, int s)`

Author: Elias Kittle

Description: This function creates a new instance of an alarm struct. Then it sets the data members for the newly instantiated instance to the relative hours, minutes, seconds, and each char to the message char array data member. The new alarm is returned.

`int alarm_add(char *msg, int h, int m, int s)`

Author: Elias Kittle

Description: Creates a new alarm and puts it into the end of a global alarm list. Used in set_alarm.

void alarm_check(void)

Author: Elias Kittle

Description: Checks the current time against all alarm clocks in the alarm linked list. If the timer is due, it prints the alarm's message and removes it from the queue. Used in the running_alarm process.

void running_alarm(void)

Author: Elias Kittle

Description: Ran as a process that calls alarm_check to print any alarms that are due, then idles the cpu. When no alarms are present, it exits. Process is started by set_alarm.

void alarm_remove(alarm *a)

Author: Elias Kittle

Description: Removes the alarm given to it from the global alarm list. Returns 0 if successful and 1 if it couldn't be found.

int set_alarm(char time[16], char *message)

Author: Elias Kittle

Description: Starts by ensuring the time is the expected HH:MM:SS format. Then it checks to see if the hour, minute, and seconds are possible. Then it calls alarm_add to add a timer to the timer queue. If the timer process isn't running, it creates and inserts one. Returns 0 if successful. Returns 1 if something went wrong somewhere and prints why if not.

R5 Functions

int alloc_mem(int size)

Author(s): Blake Mongold

Description: Allocates a block of heap memory of the requested size by calling allocate_memory(). If the allocation is successful, the function prints the starting address of the allocated memory block in hexadecimal format. If the allocation fails, an error message is printed. Returns 0 if successful or -1 if unsuccessful.

int free_mem(char *block_addr)

Author(s): Blake Mongold

Description: Frees a previously allocated block of heap memory by calling free_memory(). The input parameter is a hexadecimal string representing the starting address of the memory block to be freed. The function converts this string into an address and passes it to free_memory(). If freeing fails, an error message is printed; otherwise, a success message is printed. Returns 0 if successful or -1 if unsuccessful.

void print_mcbs(int flag)

Author(s): Elias Kittle

Description: Runs through the mcb list to print out either all the allocated or free mcbs. If flag is 1, prints allocated. If flag is 0, prints free. Used in commands show_allocated and show_free.

void print_mcb(mcb* curr_mcb)

Author(s): Elias Kittle

Description: Prints out the start of the address block and size of the given mcb. Used in print_mcbs.

void initialize_heap(size_t size)

Author(s): Michael Kaulfuss

Description: Allocates memory from the low-level kernel heap using kmalloc and adds a single large free block to a doubly linked list.

void* allocate_memory(size_t size)

Author(s): Michael Kaulfuss

Description: Allocates memory from the free list in a first fit manner, splitting free memory blocks into smaller segments only taking what is needed to fulfill allocation requests. This function moves nodes that are allocated to the front of the list.

int free_memory(void* addr)

Author(s): Michael Kaulfuss

Description: Frees a certain MCB with a given address by iterating over every element checking to see whether an allocated flag is part of the MCB and the address is correct. If it is the element's flag is changed to free, removed from the allocated list and moved to the free list where elements are checked in a loop left and right to see if there are adjacent memory blocks that are able to be merged.

char *itoh(int dec_num)

Author(s): Elias Kittle, Mason Phillips

Description: Converts a decimal number into a hexadecimal string. Divides a given decimal number repeatedly by 16, converts remainder into a hex character (by calling hex_of_int), and adds the character to the hex string. Once dec_num has been fully broken down, "0x" is prepended to the string, and it is null terminated.

int htoi(char *hex_num)

Author(s): Elias Kittle, Mason Phillips

Description: Converts a hexadecimal number string/char array, checks to ensure the hex num has the prepended '0' and 'x'. It then continuously calls the helper function return_int_of_hex, on each character to account for alphanumeric letters from A-F and 0-9. Once these are interpreted, the powerInt function is called using the converted result from return_int_of_hex function continuously multiplying by this conversion and adding it to the total.

int int_of_hex(char digit)

Author(s): Elias Kittle

Description: Helper function that converts a single hex character into the corresponding integer.

char hex_of_int(int num)

Author(s): Mason Phillips

Description: Helper function that converts an integer (0-15) into the corresponding hex character.

int power_int(int base, int power)

Author(s): Elias Kittle

Description: A helper function that takes a base and raises it to a certain power. This function works by continuously multiplying and assigning in a loop the base to result.

R6 Functions

int serial_open(device dev, int speed)

Author(s): Blake Mongold

Description: Takes a device and a speed and validates these passed parameters. It then checks to make sure the current dcb is not already open, then initializes the dcb and places it

into the dcb array. Next, based on the given speed passed, it calculates the baud rate and checks it to ensure it is valid. It then modifies the register values to indicate that the serial port is open.

int serial_close(device dev)

Author(s): Blake Mongold

Description: Takes a device parameter, performs validation checks such as ensuring the dcb is not already closed, then modifies register values to indicate that the serial port is closed.

int serial_read(device dev, char* buf, size_t len)

Author(s): Michael Kaulfuss

Description: Reads from the buffer of the device. Read until it reaches the length. If the buffer is full then return or until there is no more. Otherwise place them into the IOCB buffer instead of the ring buffer.

int serial_write(device dev, char *buf, size_t len)

Author(s): Michael Kaulfuss

Description: Transfers a block of data into a serial port. Iteratively stores bytes to a port. An interrupt enabled register ORs the value that is output at an address then adds the IER reg address. The function returns a -401 serial port not open, -402 invalid buffer address, -403 invalid count address or count value, and -404 for a busy device.

void serial_interrupt(void)

Author(s): Blake Mongold

Description: Handles interrupts generated by the serial port. The function begins by disabling interrupts to prevent interference during execution. It then determines the correct device control block (DCB) associated with the interrupting device. The Interrupt Identification Register (IIR) is read to identify the type of interrupt that occurred. Based on this value, the function calls the appropriate handler for either input or output operations. After the interrupt has been processed, an End Of Interrupt (EOI) signal is sent to the Programmable Interrupt Controller (PIC). Finally, interrupts are re-enabled before returning from the function.

void serial_input_interrupt(dcb* dcb)

Author(s): Michael Kaulfuss

Description: Handles input interrupts for the serial port. The function reads a character from the device and checks the current state of the DCB. If the DCB is in a reading state, the character is stored in the associated IOCB buffer. If the buffer becomes full or a newline character is

received, the operation is marked as complete. Otherwise, the character is stored in the ring buffer for later processing.

`void serial_output_interrupt(dcb* dcb)`

Author(s): Blake Mongold

Description: Handles output interrupts for the serial port. The function checks if the DCB is in a writing state and determines if there are remaining characters in the associated IOCB buffer. If additional data is available, the next character is written to the device. If the last character has been transmitted, the function disables write interrupts and signals completion of the operation.

`int has_unread_char(dcb* device)`

Author(s): Elias Kittle, Michael Kaulfuss

Description: Checks if the specified device control block (DCB) has unread characters in its ring buffer. The function returns 1 if there are one or more unread characters, and 0 otherwise.

`char ring_read(dcb* device)`

Author(s): Elias Kittle, Michael Kaulfuss

Description: Reads a character from the device's ring buffer. If there are no unread characters, the function returns 0 as a failsafe. Otherwise, it retrieves the next character at the current read index, increments the index using wrap-around logic, and updates the DCB to reflect one fewer unread character. The read character is then returned.

`void ring_write(dcb* device, char input)`

Author(s): Elias Kittle, Michael Kaulfuss

Description: Writes a character to the device's ring buffer. If the buffer is full then it overwrites the oldest byte.

`int loop_ring(int place, int length)`

Author(s): Elias Kittle

Description: Ensures an index stays within the bounds of the ring buffer. The function returns the remainder of the index divided by the buffer length, allowing the index to wrap around to the beginning when it reaches the end.

`void enqueue_iocb(dcb *d, struct iocb *new_iocb)`

Author(s): Michael Kaulfuss

Description: Adds the IOCB to the queue at the end. If the queue is NULL then it is inserted at the start.

`iocb* dequeue_iocb(dcb *d)`

Author(s):

Description: Removes the IOCB from the queue. If it is at the start then both head and tail pointers are set to NULL. Returns the IOCB removed.

`dcb* find_dcb(device dev)`

Author(s): Michael Kaulfuss

Description: Helper function that, based on the passed device macro (i.e. COM1), returns the dcb associated with the device.

`int dcbArrLoc(device dev)`

Author(s): Blake Mongold

Description: Helper function, based on the passed device macro (i.e. COM1), returns an index number 0-3 for the dcb_array array location.

`void IOScheduler(iocb* queued_iocb)`

Author(s): Michael Kaulfuss, Elias Kittle

Description: This function is called after sys_call makes an iocb for a read/write request. It takes that as input and if the device is free, starts a read/write. If not, it queues it into that device's dcb's iocb list.

`void IOComplete(void)`

Author(s): Elias Kittle

Description: This function is called as soon as sys_call is run. It loops through all initialized devices to check if any reads/writes have recently been completed. If so, it unblocks the process associated with the completed iocb, passes the byte amount transferred as a return value, frees the iocb, and if there's a queued iocb, it sets that up with ioscheduler.

`iocb* create_iocb(int op, dcb *curr_dcb, char *buffer, int size)`

Author(s): Michael Kaulfuss

Description: Allocates memory for a new IOCB, and sets the IOCB struct fields to the parameters passed to the function. Returns the new IOCB.

`int store_char(char* buffer, int len, char single_char)`

Author(s): Michael Kaulfuss, Mason Phillips

Description: Stores a provided character to the next open position in the provided buffer, and returns 0 upon success. If no open position is found, an error code of -1 is returned.